ChartCard.js
JavaScript

Charts.js
JavaScript

DashboardOverview.js
JavaScript

dashboardService.js
JavaScript

budgetary_quotation_mo...
JavaScript

cpds_document_model.js
JavaScript

cpds_form_model.js
JavaScript

crm_leads_model.js
JavaScript

domestic_leads_model.js
JavaScript

export_leads_history_mo...
JavaScript

export_leads_model.js
JavaScript

inhouserd_model.js
JavaScript

lead_submitted_model.js
JavaScript

lost_form_model.js
JavaScript

dashboardController.js
JavaScript

dashboardRoutes.js
JavaScript

I want all analytics to take data from the postgres tables in the database I want code for Lost lead analysis
Detailed breakdown of all lost opportunities
Give a ready to run part of code and give a step by step guide where do I add and replace what

Great — you already have **most of the system ready**, we just need to:

✅ Pull **lost leads from PostgreSQL (Sequelize models)**
✅ Create **API for lost analysis**
✅ Connect it to your **DashboardOverview table (replace mock)**

🚀 FINAL RESULT YOU WILL GET

Your **Lost Lead Analysis table** will show:

- Tender Name

- Customer
- Domain
- Value (Cr)
- Reason for loss
- Competitor
- Owner

👉 Data will come from:

- `DomesticLeadsModel`
- `ExportLeadsModel`
- OR `LostFormModel` (better structured)

🧠 STEP 1 — BACKEND (Service Layer)

📍 File: `dashboardService.js` dashboardService

✅ ADD THIS FUNCTION

```
async getLostLeadsAnalysis() {
  try {
    // Combine all sources
    const domestic = this.domesticLeads || [];
    const exportLeads = this.exportLeads || [];

    // Filter only LOST leads
    const lostLeads = [...domestic, ...exportLeads].filter(
      (lead) =>
        lead.wonLostParticipated === "Participated-Lost" ||
        lead.wonLostParticipated === "Lost"
    );

    // Format for frontend
    const formatted = lostLeads.map((lead) => ({
      tenderName: lead.tenderName,
      customer: lead.customerName,
      domain: lead.businessDomain,
      value: parseFloat(lead.estimatedValueInCrWithoutGST || 0),
      owner: lead.leadOwner,
      reason: lead.reasonForLossingOpp || "N/A",
      competitor: lead.competitorsInfo || "N/A",
```

👉 This uses fields from your model domestic_leads_model

🧠 STEP 2 — CONTROLLER

📍 File: `dashboardController.js` dashboardController

✅ ADD THIS FUNCTION

```
export const getLostLeadsAnalysis = async (req, res, next) => {
  try {
    const data = await dashboardService.getLostLeadsAnalysis();
    sendResponse(res, data);
  } catch (err) {
    next(err);
  }
};
```

🧠 STEP 3 — ROUTES

📍 File: `dashboardRoutes.js` dashboardRoutes

✅ ADD THIS LINE

JavaScript

✅ ADD ROUTE

JavaScript

🧠 STEP 4 — FRONTEND (Replace Mock Data)

📍 File: `DashboardOverview.js` DashboardOverview

❌ OLD (REMOVE THIS)

Somewhere near table rendering you likely have:

JavaScript

✅ NEW — ADD STATE

JavaScript

✅ ADD API CALL INSIDE useEffect

```
axios
  .get("http://localhost:5000/api/dashboard/lost-analysis")
  .then((res) => {
    setLostLeads(res.data.data);
  })
  .catch((err) => console.log(err));
```

✅ REPLACE TABLE BODY

Find:

```
<TableBody>
```

JavaScript

🔥 REPLACE WITH:

```
<TableBody>
  {lostLeads.map((row, idx) => (
    <TableRow key={idx}>
      <TableCell>{row.tenderName}</TableCell>
      <TableCell>{row.customer}</TableCell>
      <TableCell>{row.domain}</TableCell>
      <TableCell>₹{row.value} Cr</TableCell>
```

```
        <TableCell>{row.reason}</TableCell>
        <TableCell>{row.competitor}</TableCell>
        <TableCell>{row.owner}</TableCell>
      </TableRow>
    ))}
  </TableBody>
```

🧠 STEP 5 — (OPTIONAL BUT POWERFUL)

Add grouping (INSANE ANALYTICS 🔥)

You can extend backend:

🔹 Loss by Reason

```
const reasonMap = {};

lostLeads.forEach((lead) => {
  const reason = lead.reason || "Unknown";
  reasonMap[reason] = (reasonMap[reason] || 0) + 1;
});

const reasonBreakdown = Object.entries(reasonMap).map(([key, value]) => ({
  reason: key,
  count: value,
}));
```

👉 This can power:

- Pie chart (Why we lose deals)
- Strategy insights

🧩 WHERE EXACTLY YOU CHANGE WHAT

Step	File	Action
1	dashboardService.js	✅ Add `getLostLeadsAnalysis()`
2	dashboardController.js	✅ Add controller
3	dashboardRoutes.js	✅ Add route `/lost-analysis`
4	DashboardOverview.js	❌ Remove mock

Step	File	Action
5	DashboardOverview.js	✅ Add API call
6	DashboardOverview.js	✅ Replace table body

🔥 FINAL OUTPUT (WHAT YOU ACHIEVE)

✅ Fully dynamic PostgreSQL-driven analytics
✅ No mock data
✅ Real-time lost lead insights
✅ Scalable for charts + AI insights

⚡ BONUS (If you want next level)

I can help you add:

- 🔥 Loss Heatmap (Domain vs Reason)
- 🔥 Competitor win analysis
- 🔥 Loss trend over months
- 🔥 AI suggestions: "Why deals are lost"

If you want, next I can:
👉 Turn this into **chart (pie + bar) + KPI cards**
👉 Or integrate with your existing `Charts.js` cleanly Charts

Just tell 👍

Sources

Performance by business domain

Stacked by outcome

I want this to be changed Leads per Owner

Columns:
• Lead Owner .1
Graph Bar or any other better suiting graph

Perfect — this is a **very good upgrade** 🔥
You're moving from *Domain-based analysis → Owner performance analytics*, which is **much more useful for management dashboards**.

🎯 WHAT YOU WANT

Replace:

👉 **"Performance by business domain (Won vs Lost)"**

With:

👉 **"Leads per Owner"**

- X-axis → Lead Owner
- Y-axis → Count (or Value)
- Stacked → Won / Lost / In Progress

✅ BEST GRAPH TYPE

👉 Use **Stacked Bar Chart** (already supported in your Charts.js Charts)

Why?

- Easy comparison between owners
- Shows performance split clearly
- Matches your existing UI

🚀 STEP 1 — BACKEND (Service)

📍 File: `dashboardService.js` dashboardService

✅ ADD FUNCTION

```
async getLeadsPerOwner() {
  try {
    const domestic = this.domesticLeads || [];
    const exportLeads = this.exportLeads || [];

    const allLeads = [...domestic, ...exportLeads];

    const ownerMap = {};

    allLeads.forEach((lead) => {
      const owner = lead.leadOwner || "Unknown";

      if (!ownerMap[owner]) {
        ownerMap[owner] = {
          owner,
          Won: 0,
          Lost: 0,
          InProgress: 0,
        };
      }

      const status = lead.wonLostParticipated;
```

🚀 STEP 2 — CONTROLLER

📍 `dashboardController.js` dashboardController

✅ ADD

```
export const getLeadsPerOwner = async (req, res, next) => {
  try {
    const data = await dashboardService.getLeadsPerOwner();
    sendResponse(res, data);
  } catch (err) {
    next(err);
  }
};
```

🚀 STEP 3 — ROUTES

📍 `dashboardRoutes.js` dashboardRoutes

✅ ADD

```
import { getLeadsPerOwner } from "../controllers/dashboardController.js";
```

JavaScript

🚀 STEP 4 — FRONTEND (Charts.js)

You already have `OwnerPerformanceChart` ❗ Charts

BUT that is **single value (₹)**

👉 We need **STACKED BAR**

✅ ADD NEW CHART (IMPORTANT)

📍 `Charts.js`

```
export function LeadsPerOwnerChart({ data }) {
  return (
    <ResponsiveContainer width="100%" height={220}>
      <BarChart data={data}>
        <CartesianGrid stroke="#e4e8ef" />
        <XAxis dataKey="owner" />
        <YAxis />
        <Tooltip />
        <Legend />
        <Bar dataKey="Won" stackId="a" fill="#16a34a" />
        <Bar dataKey="Lost" stackId="a" fill="#dc2626" />
        <Bar dataKey="InProgress" stackId="a" fill="#2563eb" />
      </BarChart>
    </ResponsiveContainer>
  );
}
```

🚀 STEP 5 — FRONTEND (DashboardOverview.js)

📍 File: DashboardOverview

❌ REMOVE OLD

JavaScript

✅ ADD STATE

JavaScript

✅ ADD API CALL

Inside `useEffect`:

```
axios
  .get("http://localhost:5000/api/dashboard/owner-performance")
  .then((res) => {
    setOwnerData(res.data.data);
  })
  .catch((err) => console.log(err));
```

✅ REPLACE UI BLOCK

```
<ChartCard
  title="Leads per Owner"
  subtitle="Stacked by outcome"
  chip={{ label: "Owner view", type: "green" }}
  legend={[
    { label: "Won", color: "#16a34a" },
    { label: "Lost", color: "#dc2626" },
    { label: "In Progress", color: "#2563eb" },
  ]}
>
  <LeadsPerOwnerChart data={ownerData} />
</ChartCard>
```

🧠 FINAL DATA FORMAT (IMPORTANT)

Backend returns:

```
[
  {
    owner: "John",
    Won: 5,
    Lost: 3,
    InProgress: 7
  },
  {
    owner: "Alice",
    Won: 2,
    Lost: 6,
    InProgress: 4
  }
]
```

🔥 RESULT

You now have:

✅ Owner-wise performance
✅ Clear accountability
✅ Win/Loss tracking per person
✅ Management-ready dashboard

⚡ NEXT LEVEL (Highly Recommended)

I can upgrade this further to:

🔥 1. Value-based Owner Performance

Instead of count → ₹ Crores

🔥 2. Conversion Rate per Owner

```
Win % = Won / (Won + Lost)
```

🔥 3. Top Performer Highlight

Auto KPI:

- Best Owner
- Worst Conversion

If you want, I can next:
👉 Combine **Count + Value in same chart (dual axis)**
👉 Or add **ranking leaderboard UI** 🚀

Sources

Give ranking leaderboard

Perfect 🔥 — this is exactly what makes your dashboard **management-level powerful**.

We'll build a 🏆 **Lead Owner Ranking Leaderboard** based on:

- Wins
- Total leads
- Win % (conversion rate)
- Total value (optional upgrade)

🎯 WHAT YOU WILL GET

A leaderboard like:

Rank	Owner	Won	Lost	Total	Win %
🥇	John	12	3	15	80%
🥈	Alice	9	4	13	69%
🥉	Bob	6	6	12	50%

🚀 STEP 1 — BACKEND (Service)

📍 `dashboardService.js` dashboardService

✅ ADD THIS FUNCTION

```
async getOwnerLeaderboard() {
  try {
    const domestic = this.domesticLeads || [];
    const exportLeads = this.exportLeads || [];

    const allLeads = [...domestic, ...exportLeads];

    const ownerMap = {};

    allLeads.forEach((lead) => {
      const owner = lead.leadOwner || "Unknown";

      if (!ownerMap[owner]) {
        ownerMap[owner] = {
          owner,
          Won: 0,
          Lost: 0,
          Total: 0,
        };
      }

      const status = lead.wonLostParticipated;
```

🚀 STEP 2 — CONTROLLER

📍 `dashboardController.js` dashboardController

```
export const getOwnerLeaderboard = async (req, res, next) => {
  try {
    const data = await dashboardService.getOwnerLeaderboard();
    sendResponse(res, data);
  } catch (err) {
    next(err);
  }
};
```

🚀 STEP 3 — ROUTES

📍 `dashboardRoutes.js` dashboardRoutes

```
import { getOwnerLeaderboard } from "../controllers/dashboardController.js";
```

JavaScript

🚀 STEP 4 — FRONTEND (DashboardOverview.js)

📍 DashboardOverview

✅ ADD STATE

JavaScript

✅ ADD API CALL

Inside `useEffect`:

```
axios
  .get("http://localhost:5000/api/dashboard/owner-leaderboard")
  .then((res) => setLeaderboard(res.data.data))
```

JavaScript

🚀 STEP 5 — LEADERBOARD UI (🔥 IMPORTANT)

Add this **below charts section**

✅ ADD TABLE UI

```
<Paper
  sx={{
    border: "1px solid #e4e8ef",
    borderRadius: "14px",
    overflow: "hidden",
  }}
>
  <Box
    sx={{
      padding: "16px 20px",
      borderBottom: "1px solid #e4e8ef",
    }}
  >
    <div style={{ fontSize: "13px", fontWeight: 500 }}>
      🏆 Lead Owner Leaderboard
    </div>
    <div style={{ fontSize: "11px", color: "#8892a4" }}>
      Ranked by win rate and performance
    </div>
  </Box>

  <Table>
```

🔥 RESULT

You now have:

✅ Rank-based leaderboard
✅ Top performers highlighted
✅ Color-coded performance
✅ Works with real PostgreSQL data

⚡ NEXT LEVEL (Highly Recommended)

If you want 🔥 premium dashboard:

🧠 1. Add Value Ranking

Sort by ₹ instead of count

🧠 2. Add Filters

- FY filter
- Domain filter
- Owner filter

🧠 3. Add “Top Performer Card”

```
🏆 Best Owner: John (82%)
```

💥 PRO TIP

This leaderboard becomes **your strongest dashboard feature** during demo/interview.

If you want next:
👉 I can convert this into **animated leaderboard cards (like Power BI)**
👉 Or add **AI insights: "Why top performer is winning"**

Sources

Document type breakdown

RFP / RFQ / RFI etc.

I want code for this it must refer domestic leads table and export leads table

Perfect 👍 — this is a **clean and powerful analytics block** and fits perfectly with your existing dashboard.

You want:

🎯 Document Type Breakdown

👉 Source:

- `DomesticLeadsModel`
- `ExportLeadsModel`
  👉 Field: `documentType` domestic_leads_model

👉 Output:

- RFP
- RFQ
- RFI

- Others

🚀 STEP 1 — BACKEND (Service)

📍 `dashboardService.js` dashboardService

✅ ADD FUNCTION

```
async getDocumentTypeBreakdown() {
  try {
    const domestic = this.domesticLeads || [];
    const exportLeads = this.exportLeads || [];

    const allLeads = [...domestic, ...exportLeads];

    const docMap = {};

    allLeads.forEach((lead) => {
      let docType = lead.documentType || "Unknown";

      // Normalize (VERY IMPORTANT)
      docType = docType.toUpperCase().trim();

      if (!docMap[docType]) {
        docMap[docType] = 0;
      }

      docMap[docType]++;
    });
```

🚀 STEP 2 — CONTROLLER

📍 `dashboardController.js` dashboardController

```
export const getDocumentTypeBreakdown = async (req, res, next) => {
  try {
    const data = await dashboardService.getDocumentTypeBreakdown();
    sendResponse(res, data);
  } catch (err) {
    next(err);
```

```
  }
};
```

🚀 STEP 3 — ROUTES

📍 `dashboardRoutes.js` dashboardRoutes

```
import { getDocumentTypeBreakdown } from "../controllers/dashboardController.js";
```

JavaScript

🚀 STEP 4 — FRONTEND (DashboardOverview.js)

📍 DashboardOverview

❌ REMOVE MOCK

JavaScript

✅ ADD STATE

JavaScript

✅ ADD API CALL

Inside `useEffect`:

```
axios
  .get("http://localhost:5000/api/dashboard/document-type")
  .then((res) => {
    const colors = ["#2563eb", "#16a34a", "#d97706", "#dc2626"];

    const formatted = res.data.data.map((item, idx) => ({
      ...item,
      color: colors[idx % colors.length],
    }));

    setDocumentTypeData(formatted);
  })
  .catch((err) => console.log(err));
```

🚀 STEP 5 — USE EXISTING PIE CHART

You already have:

👉 `DocumentTypeChart` in Charts.js Charts

✅ ADD UI BLOCK

```
<ChartCard
  title="Document Type Breakdown"
  subtitle="RFP / RFQ / RFI distribution"
  chip={{ label: "All Leads", type: "blue" }}
  legend={documentTypeData.map((d) => ({
    label: d.name,
    color: d.color,
  }))}
>
  <DocumentTypeChart data={documentTypeData} />
</ChartCard>
```

🧠 FINAL OUTPUT FORMAT

Backend returns:

```
[
  { name: "RFP", value: 25 },
  { name: "RFQ", value: 18 },
  { name: "RFI", value: 10 },
```

JavaScript

Frontend converts to:

```
[
  { name: "RFP", value: 25, color: "#2563eb" },
  ...
```

JavaScript

🔥 RESULT

✅ Real PostgreSQL data
✅ Combined domestic + export
✅ Clean pie chart
✅ Dynamic categories

⚡ NEXT LEVEL (Highly Recommended)

If you want 🔥:

🧠 1. Group small values into "Others"

JavaScript

🧠 2. Add % labels

JavaScript

🧠 3. Add filter:

- FY
- Owner

- Domain

💥 PRO TIP

This chart becomes very useful when combined with:

👉 Conversion funnel
👉 Lost analysis

You'll see:

- Which document types win more
- Which ones you lose frequently

If you want next:
👉 I can build **"Win Rate by Document Type" (VERY powerful insight 🔥)**

Sources

Win Rate by Document Type
I want seperate code for this as well

Excellent 🔥 — this is a **high-value analytics metric** (used in real CRM dashboards).

🎯 WHAT YOU'RE BUILDING

📊 Win Rate by Document Type

For each:

- RFP / RFQ / RFI / etc.

You will show:

Document Type	Won	Lost	Win %
RFP	10	5	66%

Document Type	Won	Lost	Win %
RFQ	6	6	50%

🧠 LOGIC

```
Win % = Won / (Won + Lost) * 100
```

👉 Ignore:

- Not participated
- In progress

🚀 STEP 1 — BACKEND (Service)

📍 `dashboardService.js` dashboardService

✅ ADD FUNCTION

```
async getWinRateByDocumentType() {
  try {
    const domestic = this.domesticLeads || [];
    const exportLeads = this.exportLeads || [];

    const allLeads = [...domestic, ...exportLeads];

    const docMap = {};

    allLeads.forEach((lead) => {
      let docType = (lead.documentType || "Unknown")
        .toUpperCase()
        .trim();

      if (!docMap[docType]) {
        docMap[docType] = {
          documentType: docType,
          Won: 0,
          Lost: 0,
        };
      }
```

🚀 STEP 2 — CONTROLLER

📍 `dashboardController.js` dashboardController

```
export const getWinRateByDocumentType = async (req, res, next) => {
  try {
    const data = await dashboardService.getWinRateByDocumentType();
    sendResponse(res, data);
  } catch (err) {
    next(err);
  }
};
```

🚀 STEP 3 — ROUTES

📍 `dashboardRoutes.js` dashboardRoutes

```
import { getWinRateByDocumentType } from "../controllers/dashboardController.js";
```

JavaScript

🚀 STEP 4 — FRONTEND (Charts.js)

📍 Charts

✅ ADD NEW CHART

```
export function WinRateDocumentChart({ data }) {
  return (
    <ResponsiveContainer width="100%" height={220}>
      <BarChart data={data}>
        <CartesianGrid stroke="#e4e8ef" />
        <XAxis dataKey="documentType" />
        <YAxis domain={[0, 100]} />
```

```
        <Tooltip formatter={(v) => `${v}%`} />
        <Bar
          dataKey="winRate"
          fill="#16a34a"
          radius={[5, 5, 0, 0]}
        />
      </BarChart>
    </ResponsiveContainer>
  );
}
```

🚀 STEP 5 — FRONTEND (DashboardOverview.js)

📍 DashboardOverview

✅ ADD STATE

JavaScript

✅ ADD API CALL

```
axios
  .get("http://localhost:5000/api/dashboard/winrate-document-type")
  .then((res) => setWinRateData(res.data.data))
```

JavaScript

✅ ADD UI BLOCK

```
<ChartCard
  title="Win Rate by Document Type"
  subtitle="Success % per document category"
  chip={{ label: "Performance", type: "green" }}
>
```

JavaScript

🧠 FINAL OUTPUT FORMAT

Backend returns:

```
[
  { documentType: "RFP", Won: 10, Lost: 5, winRate: 66 },
  { documentType: "RFQ", Won: 6, Lost: 6, winRate: 50 },
  { documentType: "RFI", Won: 2, Lost: 8, winRate: 20 }
]
```

🔥 RESULT

✅ Real PostgreSQL data
✅ Decision-making insight
✅ Clean % visualization
✅ Sorted performance

⚡ NEXT LEVEL (VERY POWERFUL 🔥)

If you want:

🧠 1. Add VALUE-BASED win rate

Instead of count → ₹ Crores

🧠 2. Combine count + % (dual chart)

🧠 3. Insight text (AUTO AI)

```
RFP has highest success rate (66%)
RFI has lowest performance (20%)
```

💥 REAL-WORLD IMPACT

This tells management:

👉 "Which type of tenders we should focus on"

If you want next:
👉 I can combine this with **Lost Reasons → Root Cause Analysis** 🔥

Sources

Pipeline by present status

All open leads grouped by current stage
Instead of this I want this to be replaced by how many are open and how many are closed leads Pie can be used or any such type of better graph

Great decision 👍 — this simplifies your dashboard and makes it **much clearer for management**.

🎯 WHAT YOU WANT

Replace:

❌ Pipeline by present status

With:

✅ **Open vs Closed Leads**

📊 BEST GRAPH TYPE

👉 **Pie Chart (Recommended)**

- Simple

- Instantly readable
- Perfect for 2-category comparison

(You already have PieChart support in Charts.js Charts — so reuse it)

🧠 DATA SOURCE

From your models:

- `DomesticLeadsModel`
- `ExportLeadsModel`

👉 Field used:

```
openClosed
```

🚀 STEP 1 — BACKEND (Service)

📍 `dashboardService.js` dashboardService

✅ ADD FUNCTION

```
async getOpenClosedLeads() {
  try {
    const domestic = this.domesticLeads || [];
    const exportLeads = this.exportLeads || [];

    const allLeads = [...domestic, ...exportLeads];

    let open = 0;
    let closed = 0;

    allLeads.forEach((lead) => {
      const status = (lead.openClosed || "").toLowerCase();

      if (status === "open") {
        open++;
      } else if (status === "closed") {
        closed++;
      }
    });
```

```
    return [
```

🚀 STEP 2 — CONTROLLER

📍 `dashboardController.js` dashboardController

```
export const getOpenClosedLeads = async (req, res, next) => {
  try {
    const data = await dashboardService.getOpenClosedLeads();
    sendResponse(res, data);
  } catch (err) {
    next(err);
  }
};
```

🚀 STEP 3 — ROUTES

📍 `dashboardRoutes.js` dashboardRoutes

```
import { getOpenClosedLeads } from "../controllers/dashboardController.js";
```

JavaScript

🚀 STEP 4 — FRONTEND (DashboardOverview.js)

📍 DashboardOverview

❌ REMOVE OLD

```
PipelineStatusChart
```

JavaScript

✅ ADD STATE

JavaScript

✅ ADD API CALL

```javascript
axios
  .get("http://localhost:5000/api/dashboard/open-closed")
  .then((res) => {
    const colors = ["#2563eb", "#16a34a"];

    const formatted = res.data.data.map((item, idx) => ({
      ...item,
      color: colors[idx],
    }));

    setOpenClosedData(formatted);
  })
  .catch((err) => console.log(err));
```

🚀 STEP 5 — USE PIE CHART

You already have:

👉 `LeadOutcomesChart` (Pie) Charts

✅ ADD UI BLOCK

```jsx
<ChartCard
  title="Open vs Closed Leads"
  subtitle="Current pipeline status"
  chip={{ label: "Live Data", type: "green" }}
  legend={[
    { label: "Open", color: "#2563eb" },
    { label: "Closed", color: "#16a34a" },
  ]}
>
```

```
  <LeadOutcomesChart data={openClosedData} />
</ChartCard>
```

🧠 FINAL OUTPUT

Backend returns:

```
[
  { name: "Open", value: 42 },
  { name: "Closed", value: 28 }
]
```

JavaScript

🔥 RESULT

✅ Simple visualization
✅ Real DB data
✅ Immediate insight

👉 Management can instantly see:

- How much pipeline is active
- How much is completed

⚡ NEXT LEVEL (VERY USEFUL)

If you want 🔥:

🧠 1. Add KPI cards

```
Open Leads: 42
Closed Leads: 28
```

JavaScript

🧠 2. Add trend

👉 Open vs Closed over months

🧠 3. Add value-based

👉 ₹ Open vs ₹ Closed

💥 PRO TIP

This chart works best when placed:

👉 Next to:

- Conversion Funnel
- Win Rate

If you want next:
👉 I can combine **Open/Closed + Won/Lost into one powerful funnel** 🔥

Sources

3. Add value-based

👉 ₹ Open vs ₹ Closed

Perfect 🔥 — this is a **next-level insight** and much more valuable than just counts.

🎯 WHAT YOU WANT

💰 ₹ Open vs ₹ Closed Leads

👉 Instead of **number of leads**, you show:

- Total value of Open leads (₹ Cr)
- Total value of Closed leads (₹ Cr)

🧠 DATA SOURCE

From your models domestic_leads_model

Fields used:

```
openClosed
```

JavaScript

🚀 STEP 1 — BACKEND (Service)

📍 `dashboardService.js` dashboardService

✅ ADD FUNCTION

```javascript
async getOpenClosedValue() {
  try {
    const domestic = this.domesticLeads || [];
    const exportLeads = this.exportLeads || [];

    const allLeads = [...domestic, ...exportLeads];

    let openValue = 0;
    let closedValue = 0;

    allLeads.forEach((lead) => {
      const status = (lead.openClosed || "").toLowerCase();

      const value = parseFloat(
        lead.estimatedValueInCrWithoutGST || 0
      );

      if (status === "open") {
        openValue += value;
      } else if (status === "closed") {
        closedValue += value;
      }
```

🚀 STEP 2 — CONTROLLER

📍 `dashboardController.js` dashboardController

```
export const getOpenClosedValue = async (req, res, next) => {
  try {
    const data = await dashboardService.getOpenClosedValue();
    sendResponse(res, data);
  } catch (err) {
    next(err);
  }
};
```

🚀 STEP 3 — ROUTES

📍 `dashboardRoutes.js` dashboardRoutes

```
import { getOpenClosedValue } from "../controllers/dashboardController.js";
```

JavaScript

🚀 STEP 4 — FRONTEND (DashboardOverview.js)

📍 DashboardOverview

✅ ADD STATE

JavaScript

✅ ADD API CALL

```
axios
  .get("http://localhost:5000/api/dashboard/open-closed-value")
```

```
  .then((res) => {
    const colors = ["#2563eb", "#16a34a"];

    const formatted = res.data.data.map((item, idx) => ({
      ...item,
      color: colors[idx],
    }));

    setOpenClosedValue(formatted);
  })
  .catch((err) => console.log(err));
```

🚀 STEP 5 — UI (Pie Chart)

Reuse your existing:

👉 `LeadOutcomesChart` Charts

✅ ADD UI BLOCK

```
<ChartCard
  title="₹ Open vs Closed Leads"
  subtitle="Pipeline value distribution (Cr)"
  chip={{ label: "Value View", type: "green" }}
  legend={[
    { label: "Open", color: "#2563eb" },
    { label: "Closed", color: "#16a34a" },
  ]}
>
  <LeadOutcomesChart data={openClosedValue} />
</ChartCard>
```

🧠 FINAL OUTPUT

Backend returns:

```
[
  { name: "Open", value: 120 },
```

JavaScript

👉 Means:

- ₹120 Cr pipeline still open
- ₹85 Cr already closed

🔥 RESULT

✅ Financial visibility
✅ Pipeline strength insight
✅ Better decision making

⚡ NEXT LEVEL (VERY POWERFUL 🔥)

If you want to go pro:

🧠 1. Split Closed → Won vs Lost (VALUE)

JavaScript

👉 You'll know:

- ₹ actually won
- ₹ lost opportunities

🧠 2. Add KPI cards

```
Open Pipeline: ₹120 Cr
Closed: ₹85 Cr
Conversion Value: 58%
```

🧠 3. Add Trend

👉 Open vs Closed value over months

💥 PRO INSIGHT

This metric answers:

👉 “How much money is still in pipeline vs completed?”

If you want next:
👉 I can combine this into **FULL PIPELINE DASHBOARD (like Power BI)** 🔥

Sources